Tokenizer-Level Evasion and Defence in Text Safety Classifiers

APOORV MANI (250593365)

Online services often use a text safety classifier to decide whether a message should be allowed, blocked, or reviewed. The classifier does not read the characters directly: a tokenizer first turns the text into numerical tokens. That preprocessing step creates a security boundary. Small changes to text, or to the token sequence itself, can make a message look almost unchanged to a person while causing the classifier to see a different representation. Recent tokenizer attacks, and the defences proposed alongside them, are hard to compare because each is reported against a different victim, interface, threshold and idea of utility. This dissertation puts six attack variants and seven defences into one WordPiece toxicity-classification testbed: 936 attack instances and 6,552 paired attack-defence evaluations. Attack success is measured with a paired rule: an attacked input is counted only when its matching clean input still passes the same defence and remains correctly classified as toxic. This prevents a defence from receiving credit for mistakes that it causes on clean data.

TokenBreak and an adapted Adversarial Tokenization (AdvTok) each reach 96.2% undefended attack-success rate (ASR) on 156 clean-eligible toxic examples. The compatibility, reorder and homoglyph Unicode variants reach 91.0%, 88.5% and 85.9%. The invisible variant reaches 0%, because its attacked token identifiers are identical to the clean ones for this victim. Tokenizer translation is the strongest tested TokenBreak mitigation and still leaves 46.5% defended ASR. Utility then reverses part of that ranking: global characters-per-token filtering preserves only 44.8% of 600 complex legitimate inputs, and tokenizer translation lifts invisible-Unicode ASR from 0% to 13.4%. No control in this matrix is safe to adopt on its attack figures alone.

Declaration: I declare that this dissertation represents my own work except where otherwise explicitly stated.

1 Introduction

Do *paypal* and *p aypal* look the same to you? To a reader they can be identical, and to the machine reading them they are two completely different things. Boucher et al. showed that swapping one Latin letter for a Cyrillic one that renders identically was enough to make Google Translate turn “paypal” into the Russian word for “father”, and that a search query padded with 250 invisible characters returned no results at all instead of roughly 990 million [1].

Most large online platforms filter what people post using a small machine-learning model, a *text safety classifier*, which reads each message and predicts whether it breaks the rules. This dissertation is about getting past that filter without changing what a message says, and sometimes without changing how it looks on screen at all.

1.1 Background

The classifier does not read text the way a person does. Before the model sees anything, a component called a *tokenizer* cuts the string into short pieces called *tokens* and replaces each piece with a number, its *token identifier*. The word “insulting” might become the two tokens `insult` and `##ing`, and the model only ever sees the resulting list of numbers. Everything the classifier knows about a message has to survive that conversion.

Different tokenizers cut the same sentence differently. WordPiece, used by BERT-family models, starts from a fixed vocabulary and takes the longest piece it can match [14]; byte-pair encoding (BPE) learns which character pairs occur together most often and merges them repeatedly [15]; Unigram, from SentencePiece, scores several splittings and keeps the most probable [16]. The split changes what the model receives, by enough to change its behaviour [17, 18]. Tokenization is normally a modelling decision. It is also a security boundary, being the last point at which an attacker can influence the representation the classifier judges.

This is not a theoretical worry. Hosseini et al. showed a deployed commercial toxicity scorer giving low scores to abuse that had merely been misspelled [6], and Boucher et al. ran their perturbations against production services rather than laboratory models [1]. These findings show that small textual perturbations can substantially alter the decisions of deployed text-safety systems.

Research on adversarial text is considerably older than the recent interest in tokenizers, and it established the underlying problem. HotFlip showed that flipping a small number of characters can change a classifier's output [4], and TextBugger extended this to word-level edits against deployed systems [5]. Boucher et al. then demonstrated perturbations a reader cannot see at all: characters with no visible width, letters from other alphabets that look identical to Latin ones, and control characters that reverse the order in which text is displayed [1].

Two much newer attacks aim at the tokenizer specifically. TokenBreak adds characters to the visible string so that the tokenizer splits an important word into different pieces, and the model no longer recognises it [3]. Adversarial Tokenization (AdvTok) does not touch the string at all; it searches for a different, still valid, list of token identifiers that decodes back to exactly the same text [2]. Both end up at the same model, but they enter the pipeline at different points, which is why a defence tested against one tells you very little about the other.

1.2 Rationale

Two terms are used throughout, and both are meant in a narrow, practical sense. An *attack* is any change to an input that makes the classifier miss something it would otherwise have caught, while the message still means what the attacker intended. A *defence* is anything the operator places in front of the classifier to prevent that, without retraining the model: cleaning the text, rewriting it into a standard form, checking the token identifiers, or refusing the input outright.

The defender's problem is one of comparison rather than invention. Suppose an operator reads about a defence that stops TokenBreak. They still cannot tell whether it does anything about AdvTok, because the two papers use different classifiers, count success differently, and test on different data. They also cannot tell what the defence costs, because a control that rejects suspicious-looking text will also reject some perfectly ordinary text, and that cost is rarely reported. Putting the published numbers side by side does not help: any ranking would mostly reflect the differences between the experiments rather than the differences between the defences.

This dissertation addresses that by running the attacks and the defences against a single classifier under a single set of rules, so the comparison means something.

1.3 Novelty and Innovation

The contribution of this dissertation lies primarily in what becomes visible when previously separate attacks and defences are reproduced and evaluated under one controlled protocol. The findings below are specific to the implemented victim and threat model unless stated otherwise.

A defence-induced cross-attack regression. Invisible Unicode has no effect on the undefended classifier, yet after tokenizer translation it succeeds on 13.4% of clean-retained inputs. The defence therefore introduces a harmful interaction with an attack that was ineffective against the original pipeline. This does not show that tokenizer translation is generally unsafe; it shows why a defence should be regression-tested against attacks beyond the one it was designed to mitigate.

A paired evaluation rule that separates robustness from defence-induced damage. An apparent fall in attack success can be misleading if the defence has already caused the corresponding clean input to be rejected or misclassified. The evaluation therefore requires the clean message to survive the same defence before its attacked counterpart contributes to defended ASR. For tokenizer translation, this distinction is substantial: only 127 of 156 clean toxic examples remain correctly classified, corresponding to an 18.6% loss of previously correct decisions.

Implementation and transfer issues exposed by reproduction. Close reproduction identified two issues that required explicit handling. TokenBreak's printed procedure raises a confidence threshold before re-entering a routine that resets that threshold, so the intended escalation is lost unless the implementation carries it forward [3]. Separately, the five-token CPT window illustrated by Broken-Token does not transfer cleanly to this WordPiece configuration: 14.46% of ordinary calibration text

falls on the statistic's minimum score. The final implementation therefore uses a disclosed 10-token adaptation rather than treating the source parameter as universally applicable [8].

Resolving one proposed explanation for invisible-character failure on this victim. Boucher et al. observed that invisible characters sometimes had no effect on deployed toxicity services and proposed multiple possible explanations, including preprocessing or tokenizer behaviour [1]. In this classifier, the clean and attacked token-identifier sequences are identical for all 156 tested inputs. This identifies preprocessing as the immediate explanation for the failure of this attack in the present pipeline, without making a claim about the proprietary systems examined by Boucher et al.

Separating defence mechanism from defence outcome. Each of the 42 attack–defence combinations records not only whether attack success changes, but whether the defence blocks, rewrites, re-encodes, or leaves the input unchanged. This distinguishes apparently similar security outcomes that arise through different mechanisms and therefore carry different deployment implications.

A cost measurement borrowed from an older field. In 2000 Axelsson argued that an intrusion detector with good recall is still useless if it cries wolf too often [13]. Measuring that on code, web addresses and five non-Latin writing systems turns the warning into a number: a filter tuned to reject 1% of English prose rejects every non-English input tested.

The study began as an attempt to build a *detector* from disagreement between tokenizers. That idea was dropped for the reasons in Section 5.4, but the thread survives, since tokenizer translation is itself a cross-tokenizer mechanism and the harmful result above is a cross-tokenizer effect.

1.4 Scope and Delimitations

The primary victim is a single DistilBERT toxicity classifier with an uncased WordPiece tokenizer, used for prediction only and never retrained. The attacker wants a *false negative*: an input the clean classifier calls toxic but the attacked pipeline does not. TokenBreak and the Unicode variants submit modified text through an ordinary text interface. AdvTok assumes a lower-level interface in which token identifiers reach the model directly, which is a stronger assumption than a normal text-only application programming interface (API) and is stated wherever the result is used.

Model extraction, poisoning, availability attacks, downstream large language model (LLM) behaviour and defence-aware adaptive attacks are out of scope. A guard-model baseline is excluded structurally: the seven controls here sit in front of the classifier and leave its decision rule intact, whereas a guard model replaces that decision and would need its own eligible set and calibration before joining the matrix. A supporting experiment records clean eligibility for BPE and Unigram setups; no attack result is claimed for either.

1.5 Aim, Research Questions and Objectives

The aim is to establish, in one controlled testbed, how effectively tokenizer-level defences reduce tokenizer-level evasion of a text safety classifier, and what they cost the legitimate traffic passing through the same pipeline.

That aim is answered through three research questions:

> **RQ1.** How effective are the six tokenizer-level attack variants against the undefended classifier, and is that effectiveness explained by inputs the classifier was already unsure about?
> **RQ2.** Which of the seven defences reduce which attacks when clean and attacked inputs are processed under the same defence, and how large and how certain are those reductions?
> **RQ3.** What do those defences cost on legitimate input, measured as blocking, changed decisions and changed representations?

Each objective below was written as a checkable artefact or measurement, so that completion can be judged against the frozen evidence rather than against an open-ended implementation goal. O1 and O5

serve RQ1, O2 and O3 serve RQ2, O4 serves RQ3, and O6 serves all three. Section 5.2 records where each criterion was met.

Table 1. Objectives, measurable success criteria, and the research question each serves.

ID	Objective and success criterion
O1	Six attack variants validated on one shared eligible set: all run on the same 156 rows, with validity checks, documented deviations, success rates and probability shifts.
O2	Seven defences behind one evaluation interface, with fragmentation thresholds calibrated and frozen from clean data only.
O3	All 42 attack-defence cells evaluated with matched clean controls, defended success conditional on clean retention, every reduction carrying a 95% confidence interval (CI).
O4	Legitimate-input cost quantified: 5,250 ordinary clean controls and 600 complex inputs under all seven defences.
O5	Attack success stratified over three clean-confidence bins with 95% intervals, testing whether the headline rate is an artefact of borderline messages.
O6	Evidence trail kept auditable: frozen outputs, environment snapshot, checksum manifest, correction log, and the clean-only BPE/Unigram groundwork.

1.6 Alignment with the CyBOK

The project sits inside the CyBOK *Adversarial Behaviours* knowledge area and the supplementary *Security and Privacy of AI* knowledge guide: it specifies attacker goals and capabilities, implements evasion attacks against a deployed-style control, measures defensive effectiveness, and reports residual risk [32, 33]. There is also a direct *Software Security* connection, since canonicalisation, sanitisation and input validation all sit on the boundary between untrusted input and model execution [34].

The alignment shapes the design, not just the topic. Treating the tokenizer as a security-relevant component is what motivates the canonicality and normalisation defences, and the utility analysis is the input-validation question a CyBOK-literate reviewer would ask next: what does this control do to traffic that was never hostile?

1.7 Organisation

Section 2 reviews the attack, defence and evaluation literature and derives the requirements the artefact was built to satisfy. Section 3 covers design, data, analysis, tooling, implementation and ethics. Section 4 reports the primary results, Section 5 evaluates them against the research question and the threats to validity, and Section 6 concludes. The remaining figures sit in Appendix C at readable size and are referenced where they are discussed.

Acknowledgements

I thank my supervisor, Dr. Mujeeb Ahmed, for his guidance throughout this project, for his encouraging guidance and insightful questioning. His advice was instrumental in steering this project toward a rigorous, empirical comparative evaluation of tokenizer defences.

2 Literature Review

2.1 Tokenization and adversarial text

BPE learns merge operations over frequent symbol sequences [15]. WordPiece segments under a vocabulary constraint and is tied to BERT-family models [14]. SentencePiece trains on raw text and supplies a Unigram language-model tokenizer [16]. These are not interchangeable serialisation schemes: their segmentation and preprocessing decisions change downstream model behaviour [17, 18].

Earlier character-level adversarial work established the underlying problem. HotFlip selects character- and word-level substitutions using gradient information [4], TextBugger combines character- and word-level perturbations against deployed systems [5], and Pruthi et al. address adversarial misspellings using a word-recognition stage before classification [7]. Together, these studies show that small textual changes can substantially alter model behaviour. However, they do not directly evaluate how tokenizer-specific defences behave across different tokenizer-level attack families. That comparison is where this project begins.

2.2 Unicode perturbations

Boucher et al. are the peer-reviewed basis for the Unicode family used here [1]: invisible characters, homoglyphs, and bidirectional (Bidi) reordering, which uses invisible control characters to change the order text is displayed in. This project reproduces those three in spirit, swaps the paper's deletion family for a compatibility-character extension labelled as such, and changes the objective from disrupting sequence output to minimising the toxic-class probability, because the target here is a binary classifier.

Unicode Technical Standard #39 (UTS #39) supplies confusable and mixed-script data [9]. It is a normative standard, not an attack study, and it is used narrowly: selected one-codepoint mappings and compatibility normalisation, tested as implementation choices. Nothing here says multilingual text can safely be folded down to ASCII, and Section 4.5 gives the measured reason for that.

2.3 Tokenizer-focused attacks

TokenBreak, a recent preprint, inserts characters that change subword segmentation [3]. Its *BreakPrompt* procedure scores words in isolation and tries prefixes until a word reads as confidently benign, and it proposes tokenizer translation as the fix. Not being peer reviewed, it is used here as a specification rather than as settled evidence.

AdvTok, at ACL 2025, studies adversarial non-canonical tokenizations in a BPE and language-model setting [2]. The adaptation here moves the greedy search to a WordPiece classifier, minimises $P(\text{toxic})$, samples up to 128 neighbours per iteration and starts from a random valid segmentation. Because that is a substantial change, every output is checked to decode to the intended string, differ from the canonical segmentation, and contain no special tokens.

2.4 Defence families

The seven controls fall into three groups. Canonical reject and canonical replace compare a supplied representation against the victim tokenizer's canonical one. Three transformations alter the text or tokenization path before model inference: tokenizer translation, Unicode sanitisation, and Normalisation Form KC (NFKC) compatibility normalisation combined with targeted confusable repair. Global and windowed characters-per-token (CPT) score and block [8]. All seven operate before model inference and leave the classifier's decision rule unchanged, which is what excludes a guard model from this matrix.

Grouping them this way exposes something the source papers do not say out loud. Several of the transformation-based defences can be understood as forms of canonicalisation: they map multiple possible representations towards a standard form before classification. Such canonicalisation is safest when the mapping preserves distinctions that matter to legitimate users. Where it does not, the defence can introduce errors of its own, which is exactly what Section 4.7 measures.

Broken-Token proposes CPT as a cheap signal for obfuscated prompts. Its experiments are BPE-focused and its thresholds are chosen using both clean and obfuscated distributions. This project changes both conditions on purpose: CPT is transferred to WordPiece, and the threshold is fitted to clean WikiText alone, so no attack data informs the operating point. The published five-token window did not survive the transfer. Under this WordPiece configuration it collapsed into a score floor during calibration (Appendix C, Figure 8), so the final window is 10 tokens. That is a project adaptation and is reported as one, not as a finding about the source method.

2.5 Evidence quality

Two of the central methods here, TokenBreak and Broken-Token, are recent preprints, while AdvTok and the Unicode work carry peer-reviewed conference evidence. Since the artefact reimplements all four, provenance is part of correctness rather than a courtesy, and Table 2 records what each source can and cannot support.

Table 2. Evidence quality and role of the principal sources.

Work	Evidence status	Role here, and the boundary it carries
Boucher et al. [1]	IEEE S&P 2022, peer reviewed	Basis for the invisible, homoglyph and reorder variants; objective and one family are adapted here.
AdvTok [2]	ACL 2025, peer reviewed	Basis for the non-canonical tokenization attack; published setting is BPE/LLM, so the WordPiece classifier is an adaptation.
TokenBreak [3]	2025 preprint	Attack specification and the tokenizer-translation defence; not peer reviewed, and one pseudocode repair is disclosed.
Broken-Token [8]	2025 preprint	Motivates global and windowed CPT; BPE-focused, so threshold and window are re-derived here.
UTS #39 [9]	Unicode normative standard	Confusable and mixed-script mechanisms; not evidence that arbitrary Unicode rewriting is semantically safe.

2.6 User needs and evaluation requirements

Robustness methodology warns against judging a defence only by a fixed, non-adaptive attack [10, 11], and Papernot et al. argue for stating attacker knowledge and capability explicitly [12]. This project does not meet the stronger adaptive standard; that gap is stated here and revisited in Section 5.3. Within the frozen-attack scope it does control two routine sources of over-optimism: unmatched denominators, and utility measured only on easy English prose.

The stakeholders here are not survey participants, so their needs come from the literature. An operator needs to know which controls cover which perturbations and what legitimate traffic they reject; an evaluator needs stable denominators, intervals and explicit exclusions; legitimate users need a deployment that does not treat code, web addresses (URLs), multilingual text or mixed scripts as hostile. CheckList argues for behavioural testing beyond aggregate accuracy [19], toxicity research documents subgroup-dependent error [20, 21], and Axelsson explains why a detector with useful recall can still be unusable if it cries wolf [13]. Table 3 turns these into requirements.

Table 3. Evidence-derived security and evaluation requirements.

ID	Requirement	How it is operationalised
R1	Heterogeneous coverage	Common 6×7 matrix; cells expected to fail are executed and reported.
R2	Clean correctness	Defended success is conditioned on the matched clean message surviving the same defence.
R3	Legitimate utility	5,250 ordinary controls plus 600 complex inputs across six categories.
R4	Reporting validity	Flagged, blocked, transformed and same-label are recorded separately.
R5	Evidence provenance	Source status, deviations, frozen outputs and superseded results documented.
R6	Reproducibility	Fixed seeds, clean-only calibration, environment snapshot, checksums.

The reviewed studies evaluate attacks and defences under different victims, datasets, interfaces and evaluation procedures, making their reported results difficult to compare directly. This project addresses

that comparability gap by evaluating the selected attacks and defences under one victim, one eligibility rule, one paired protocol and one utility stress test.

3 Methodology

3.1 Research Design

This is an empirical security evaluation, not a training experiment. Every attack is generated against one frozen WordPiece classifier and replayed through the same seven defences, which removes the between-paper confounds that make published results hard to rank. Figure 1 separates that from the clean-only supporting comparison, which establishes denominators for future work and nothing else.

Threat model in plain terms. The attacker here can send messages and see how confident the classifier is, but cannot see inside the model or retrain it. Their goal is narrow: take a message the classifier correctly calls toxic and get the same meaning past it. The defender may clean, rewrite or reject an input before the classifier sees it, and may not change the classifier. Table 4 sets it out in full.

Table 4. Threat model for the primary evaluation.

Element	Specification
Adversary goal	Cause a false negative on an input the clean victim classifies as toxic.
Knowledge	Query access to victim confidence scores and tokenizer behaviour; no gradient access is used.
Ingress	TokenBreak and Unicode submit modified text, tokenized by the deployment. AdvTok supplies a non-canonical token sequence for the intended string, which assumes a lower-level interface than a string-only API.
Defender	Pre-model transformation, canonicality checking or a reject policy; the victim is never retrained.
Budget	TokenBreak follows the prefix search; Unicode uses an edit budget of five; AdvTok uses at most 25 iterations with up to 128 sampled neighbours each.
Out of scope	Defence-aware adaptive attacks, poisoning, extraction, availability attacks, downstream generative-model behaviour.

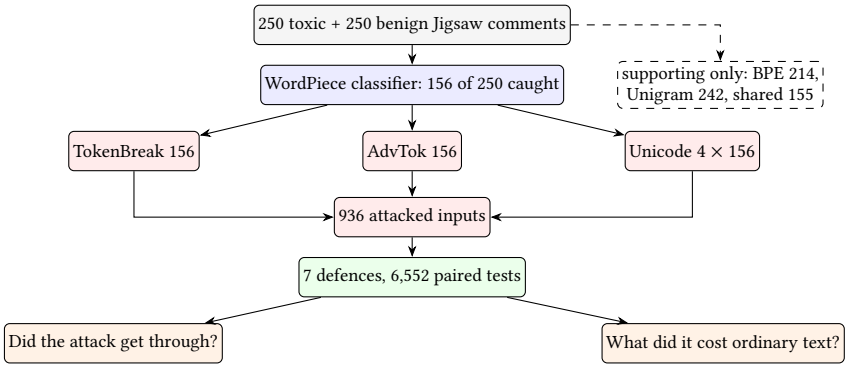

Fig. 1. How the study fits together. The 156 toxic comments the classifier catches become the test set for every attack; those give 936 attacked inputs, each run through all seven defences and judged twice, once for whether the attack still worked and once for what the defence did to ordinary text. The dashed box is a clean baseline for two other tokenizers, with no attacks run.

3.2 Data Collection

The victim is `martin-ha/toxic-comment-model`, a DistilBERT classifier with an uncased WordPiece tokenizer and binary output [24, 30], run in inference mode at a toxic threshold of 0.5. Direct token-ID sequences are capped at 510 non-special tokens so that `[CLS]` and `[SEP]` fit inside the 512-token limit.

The attack data are 250 toxic and 250 benign Jigsaw comments [31], sampled with seed 42 after an eight-word minimum. The classifier detects 156 of the 250 toxic rows and produces eight false positives; those 156 form the eligible set. Conditioning on clean positives is what makes the evasion question answerable, and it stops a pre-existing false negative counting as an attack success.

WikiText-2 is split into 5,000 calibration and 5,000 held-out lines [25]. Only the calibration split sets CPT thresholds; the held-out split is reserved for utility. The calibration target is roughly 1% blocking, giving a frozen global threshold of 2.9409 characters per token and a windowed threshold of 1.1000 over 10 tokens.

Legitimate inputs come in two layers. The ordinary controls are the 250 benign Jigsaw comments plus the 5,000 held-out WikiText lines. The stress set adds 600 inputs, 100 each of Python code, source URLs, emoji-bearing comments, deterministic single-typo variants, non-English sentences spanning five writing systems, and constructed bilingual mixed-script strings; CodeSearchNet, GoEmotions and FLORES supply the external material [26–29]. The stress set is not meant to estimate production prevalence. It exists to exercise the preprocessing paths that English prose leaves untouched.

3.3 Data Analysis

The central rule is simple. For every attack and defence, two inputs are tested: the attacked message and the original it was made from, both through the same defence. The attack counts as a win only if the original still gets through and is still called toxic, and the attacked version gets through and is called safe. The second condition matters. A defence that blocks everything lets no attack past, so a naive score calls it perfect when it has in fact broken the classifier; the same thing happens more subtly whenever a defence damages correct decisions, and the damage then reads as the attacker failing. Algorithm 1 states the procedure.

Writing C_i^d for "the original message i survives defence d" and A_i^d for "the attacked version gets through and is called safe", an attack succeeds when $S_i^d = C_i^d \wedge A_i^d$, and the rate is counted only over messages that survived:

$$\mathrm{ASR}_d = \frac{\sum_i S_i^d}{\sum_i C_i^d}.$$

The undefended comparison is recalculated over exactly the same surviving messages, so the two numbers are always about the same inputs. Algorithm 1 states the procedure in full.

Algorithm 1: Paired evaluation. A clean message the defence blocks, or causes the classifier to misjudge, is dropped from the denominator, so no defence earns credit for damage it did itself.

Input: clean messages X, attacked counterparts X', defence d, classifier f

$kept \leftarrow 0$; $won \leftarrow 0$
foreach $(x, x') \in (X, X')$ **do**
  $(a_c, r_c) \leftarrow d(x)$ `// allowed?, and what is passed on`
  if $\neg a_c \vee f(r_c) \neq$ toxic **then continue**
  $kept \leftarrow kept + 1$; $(a_a, r_a) \leftarrow d(x')$
  if $a_a \wedge f(r_a) =$ non-toxic **then** $won \leftarrow won + 1$
end
return $won/kept$

Each cell reports the surviving count, the attack's success with and without the defence, the difference, and a 95% interval from 5,000 resamples [22]. Cells are *complete* when a working attack falls to zero, *partial* when it drops but survives, *harmful* when the defence makes it work better, otherwise *no clear effect*. An attack that never worked is *no base attack*, never *complete*: no credit for stopping a non-threat. Every evaluation also records what the defence did, whether nothing, blocked, rewrote, or both, since outcome and cause differ.

The 156 messages are split by how sure the classifier was beforehand, with 95% Wilson intervals [23]. Cost is how often a defence blocks legitimate text, changes the verdict on it, or rewrites it, always against the same text with no defence.

3.4 Tools & Technologies

The artefact is Python 3.11.3 with PyTorch 2.6.0+cu124, Transformers 5.14.1, Tokenizers 0.22.2, pandas, NumPy and Matplotlib. Fast tokenizer backends serve the supporting BPE and Unigram setups, so the artefact needs no separate SentencePiece runtime. CUDA was available at the evidence freeze. The final statistical and figure scripts read frozen CSV artefacts and never call the model again, so every reported number can be regenerated without re-running an attack.

3.5 Software Design & Implementation

The pipeline is staged rather than monolithic. Attack generators write one shared result schema holding text, probabilities, token sequences and identifiers, validity fields, success labels and latency; the defence layer reads those frozen artefacts through a single interface and records whether each input was allowed, blocked or transformed. That separation paid for itself, because a downstream analysis error could be fixed without regenerating expensive attacks. The complete project artefact and source code are available in Appendix D.

3.5.1 Attack implementation and attribution. Table 5 states what came from the literature and what was changed here.

Table 5. Where every attack and defence came from, and what was changed here.

Component	Source	Implementation here, and the deviation
Attacks		
TokenBreak	BreakPrompt [3]	Word-level scoring, prefixes A–Z then a–z, base confidence 0.995. Uses a disclosed bounded deep-check repair, because the printed pseudocode resets the threshold it just raised.
AdvTok	Greedy search [2]	Minimise $P(\text{toxic})$, 25 iterations, up to 128 neighbours, random valid start. Moved from BPE/LLM to WordPiece classification.
Invisible	Boucher et al. [1]	Differential evolution, population 32, 10 iterations, budget 5; zero-width space, non-joiner and joiner. Classifier objective replaces output disruption.
Homoglyph	Boucher et al. + UTS #39 [1, 9]	Same optimiser, targeted confusable substitutions; one-codepoint mapping family only.
Reorder	Boucher et al. [1]	Bidi sequence reorders adjacent characters; the sanitiser rejects Bidi controls rather than deleting them.
Compatibility	Project extension	NFKC-compatible characters, same optimiser and budget. Not presented as a Boucher et al. family.
Defences		
Tokenizer translation	TokenBreak [3]	Segment with an XLNet/Unigram reference tokenizer, map pieces into the WordPiece vocabulary, lowercase to match the victim.
Canonical reject / replace	AdvTok [2]	Reject identifiers differing from the victim's canonical ones, or replace them and record the action.
Unicode sanitiser	Boucher et al. [1]	Reject Bidi controls, strip selected other control characters, re-tokenize.
NFKC + confusables	UTS #39 [9]	NFKC plus targeted one-codepoint repair; not a full UTS #39 skeleton.
Global / window CPT	Broken-Token [8]	Block below 2.9409 characters per token overall, or below 1.1000 in the weakest 10-token window.

One defect is worth stating outside the table. TokenBreak's published procedure keeps raising a confidence threshold when a word resists attack, then calls itself again – and that call begins by setting

the threshold back to its starting value, so every increase is discarded. Implemented exactly as printed, the escalation never happens. This artefact uses a bounded repair that keeps the raised threshold, and the deviation is disclosed here and in Table 5 rather than folded silently into the results. The implemented repair is given explicitly in Appendix B, Algorithm 2.

AdvTok carries four validity checks. The reimplemented canonical WordPiece reconstruction is first verified against the victim tokenizer. Each final output is then required to decode to the intended string, differ from the canonical identifier sequence, and contain no special tokens. All 156 final AdvTok outputs passed these checks.

3.5.2 Defence implementation and attribution. Every defence implements the same contract: text and, where relevant, supplied identifiers map to model-bound identifiers, a flag or block outcome, and an action record. The action record exists for auditability and is not offered as a separate contribution.

3.5.3 Code documentation and attribution. Each module carries a header stating what it does, which paper it implements, and where it departs from that paper, so any line can be traced to its source. Reimplemented algorithms name the original authors at the point of implementation, and every project change is marked in the code as well as in Table 5.

3.5.4 Validation and reproducibility. Only corrected artefacts feed the final results. Six changes altered the evidence: the first TokenBreak implementation was replaced once checked against the paper; AdvTok gained four validity checks; the fragmentation statistic was corrected to characters divided by tokens; the window grew from five tokens to ten after calibration exposed the score floor; deletion of Bidi controls became rejection; and the unpaired matrix was rebuilt as the paired one. Superseded outputs are excluded and kept in the audit trail.

The reproducibility freeze records the package versions above, CUDA availability, and SHA-256 hashes for scripts, datasets, results and audit files. That fixes the final evidence state. It does not prove every historical run used an identical environment, and it is not offered as if it did.

3.6 Ethical Considerations

The study uses publicly distributed datasets and offline open models. No human participants were recruited and no live third-party service was attacked; every attack ran locally against the frozen research victim.

The work is dual use: reimplementing public evasion methods lowers the effort to reproduce them, while the same artefact hardens defensive systems. The dissertation reports aggregate outcomes and methodology, reproduces no unnecessary toxic examples, and attributes every source algorithm where it is implemented.

The utility analysis shaped the evaluation rather than following it. A filter that blocks every tested non-English input places an unevenly distributed burden on legitimate multilingual users, and reporting only its attack suppression would conceal that, so legitimate-input cost sits in the main evaluation. Toxicity classification carries known subgroup and annotation limits [20, 21], and this victim's own model card discloses that it misjudges benign text [30], so these findings describe the classifier and pipeline, not the people in the dataset.

4 Results

Sections 4.1 and 4.6 answer RQ1, Sections 4.2 to 4.7 answer RQ2, and Section 4.5 answers RQ3.

4.1 Clean baseline and undefended attacks

The clean classifier detects 156/250 toxic comments (62.4% recall) and produces eight false positives on 250 benign ones (3.2%). Those 156 rows are the eligible set for every attack in Table 6; the corresponding attack-success rates are visualised in Appendix C, Figure 3.

Table 6. Undefended attack results on the common 156-row eligible set.

Attack	Success	ASR	Mean drop	Note
TokenBreak	**150/156**	**96.2%**	0.8081	Mean 9.2 words modified; deep check escalated on 12/156 rows.
AdvTok	**150/156**	**96.2%**	0.7737	All 156 outputs passed the non-canonical validity checks.
Compatibility	142/156	**91.0%**	0.7806	Project extension using NFKC-compatible characters.
Reorder	138/156	**88.5%**	0.7633	Bidirectional control-character perturbation.
Homoglyph	134/156	**85.9%**	0.7483	Targeted confusable substitution.
Invisible	0/156	**0.0%**	≈ 0	Attacked token IDs identical to clean IDs for this victim.

TokenBreak and AdvTok tie on the headline figure without being equivalent attacks: TokenBreak rewrites the string and changes several word segmentations, while AdvTok changes no visible character and supplies a different valid token sequence. The decomposition matters. Random non-canonical initialisation alone crossed the boundary on 82 of 156 examples, and the greedy search added the other 68, so quoting only the final 96.2% would credit the optimiser with more than half an effect it did not produce.

The invisible variant failed for an instructive reason. Its perturbations are present in the attacked strings, yet all 156 attacked identifier sequences match their clean counterparts, so the attack never arrives in a different representation. That 0% is a property of this preprocessing path, not of the perturbation.

This experiment also distinguishes between the explanations proposed by Boucher et al. for this victim. Boucher et al. observed that invisible characters had no effect on IBM's toxic-content classifier or Google's Perspective service, and proposed two possible explanations: either such characters had been encountered during training, or the tokenizer disregarded them [1]. They could not distinguish the two from outside the model. For this classifier, comparing the token identifiers distinguishes the two explanations: the attacked and clean identifier sequences are identical, showing that the perturbation is removed before it can alter the model input.

4.2 Attack-defence coverage

All 42 cells were evaluated and are shown in Figure 2. The frozen classification holds 9 complete suppressions, 7 partial reductions, 25 cells with no clear paired effect, six of them against an attack with no working baseline, and 1 harmful interaction.

No defence dominates. Canonical reject and replace suppress AdvTok in the low-level interface and do nothing measurable to the string attacks; NFKC repair suppresses the compatibility and homoglyph variants; the sanitiser suppresses reorder; global CPT is sensitive to AdvTok fragmentation and much weaker against TokenBreak.

Those zero cells need a qualification that changes what they are worth. The homoglyph attack and the confusable repair draw on the same mapping family, and the compatibility attack was built from characters chosen for their NFKC behaviour. They confirm the implementations do what they claim and say nothing about characters outside the tested sets, so I report them as implementation checks.

4.3 TokenBreak is the residual coverage gap

Appendix C, Figure 4 sets the seven defences side by side. Tokenizer translation keeps 127 of 156 clean toxic rows; on that subset TokenBreak falls from 95.3% to 46.5%, a 48.8 point reduction (95% CI 40.2–57.5) that still leaves close to a coin flip. Global CPT keeps 149 rows and moves 96.0% to 84.6%, an 11.4 point reduction (95% CI 6.7–16.8). The other five show no clear reduction at all.

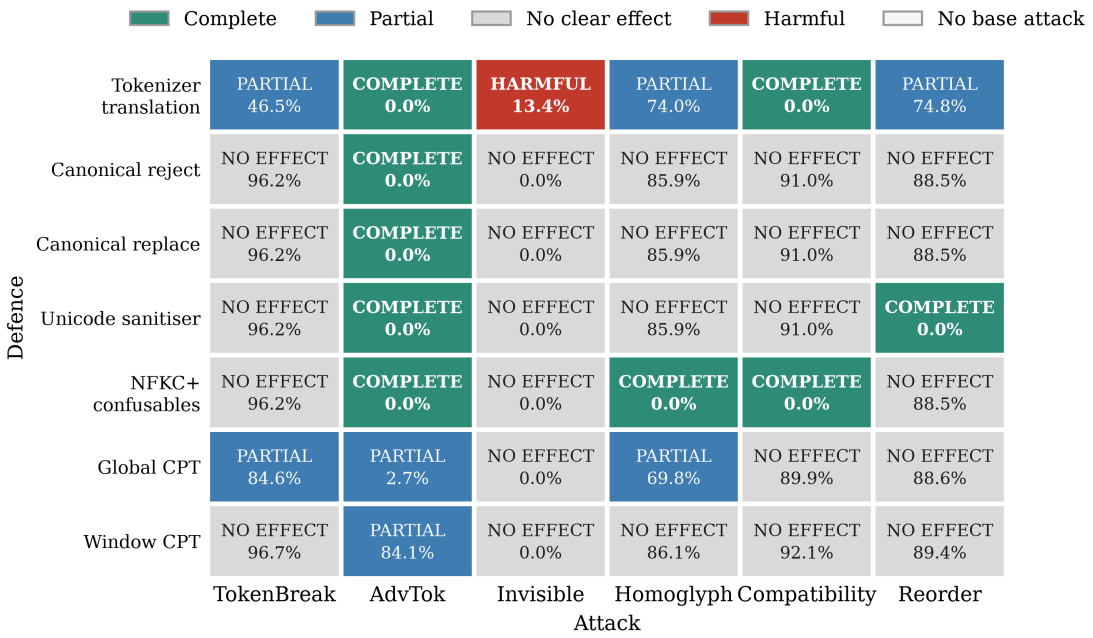

Fig. 2. Every attack against every defence. Each cell gives a status label and the defended attack success rate, so the reading does not depend on colour. Green marks a cell reduced to zero in this experiment, blue a real but partial reduction, grey no measurable effect, and red the single case where adding the defence made an attack work.

4.4 How good are the detector-style defences?

Some controls transform nothing; they score an input and block it. A *detector* that catches everything by blocking everything is useless, so recall alone will not do.

Global CPT detects one attack well and the rest poorly. On AdvTok it reaches 0.955 recall at 0.943 precision, an F1 of **0.949**. At the same threshold it falls to 0.254 F1 on TokenBreak, 0.316 on homoglyph, 0.059 on compatibility, 0.024 on invisible characters, and catches no reorder attack at all. Windowed CPT manages 0.261 on AdvTok and 0.061 on TokenBreak.

The exact controls do better in their own lane, which is what exactness buys: canonical enforcement scores 1.0 throughout on AdvTok with no false alarms, the sanitiser 0.997 F1 on invisible characters and on reorder, NFKC repair 0.975 on compatibility and homoglyph. Those are the attack-aware pairings of Section 4.2, so they measure implementation correctness, not coverage.

One pairing is worth pausing on. The sanitiser catches the invisible-character attack with perfect recall even though that attack does nothing at all to this classifier: it is flagging something real and harmless. A deployment reading flag counts as a risk signal would be counting an attack that cannot work here.

False alarms on clean text stay low: global CPT fires on 3.6% of benign comments and 1.04% of WikiText lines, windowed CPT on 1.6% and 1.04%, NFKC repair on 3.2% and 0.84%, the sanitiser on 0.4% and 0.08%, canonical enforcement on nothing. Section 4.5 is where that reassurance breaks.

4.5 Utility changes the ranking

Appendix C, Figure 5 summarises the legitimate-input effects across all seven defences. The strongest security result is not the best deployment policy. Global CPT drives AdvTok to 2.7% and preserves only 44.8% of complex legitimate input, blocking every tested web address and non-English input, 96% of mixed-script, 18% of code, 12% of emoji and 5% of misspellings. Window CPT is gentler at 63.8%, still blocking 49% of code and 80% of non-English and mixed-script text.

Clean calibration predicts none of it. Global CPT blocks about 1% of held-out WikiText, matching its target, but 3.6% of benign comments and far more of the structured and multilingual set, so the threshold is domain-dependent before an adaptive attacker is considered.

NFKC repair fails differently, preserving every decision across the 600 complex examples while rewriting 39% of non-English and 37% of mixed-script inputs. Tokenizer translation preserves 98.7% of decisions and rewrites essentially every code, URL, misspelling, non-English and mixed-script input. Whether that is acceptable depends on what sits downstream: fine for a binary verdict, possibly not for a task that consumes the text afterwards.

4.6 Confidence-margin analysis

High success could just mean the attacks flip messages that were already borderline. The eligible set rules that out (Appendix C, Figure 6):: ten fall in $(0.5, 0.6]$, 18 in $(0.6, 0.8]$ and 128 in $(0.8, 1.0]$. Across those 128, TokenBreak and AdvTok each succeed on 123 (96.1%, Wilson 95% CI 91.2–98.3 [23]), compatibility on 115, reorder on 112, homoglyph on 108, and invisible Unicode on none. The two lower bins are too small for fine ranking.

4.7 Defence-induced regression

Tokenizer translation produces the single harmful cell. Invisible Unicode has 0% undefended success, yet across the 127 clean-retained pairs it reaches 13.4%, a rise of 13.4 points (95% interval +7.9 to +19.7). The defence introduced a harmful interaction: an attack with 0% success against the undefended pipeline reached 13.4% success after tokenizer translation. The mechanism depends on this victim and reference-tokenizer pairing, so it says little about tokenizer translation in general; what it argues for is regression testing, because swapping the front-end tokenizer changes which perturbations survive into the representation.

4.8 Supporting baselines and processing cost

The same 250 toxic comments were also scored under two further model-tokenizer setups. Clean eligibility was 156/250 for the WordPiece victim, 214/250 for the BPE model and 242/250 for the Unigram model, with 155 eligible under all three. Architecture and tokenizer family change together here, so these are clean baselines only and support no claim about tokenizer-family robustness or attack transfer.

Defence processing is cheap next to attack generation. Median per-input defence latency ranged from 0.13 ms for canonical enforcement to 0.46 ms for tokenizer translation, against median attack search times of 97.6 ms for TokenBreak, 603.6 ms for AdvTok and 170–1,106 ms across the Unicode variants. These are local measurements, not throughput estimates, but the reading survives: no control here is expensive enough for compute to be the reason to reject it, so the deciding factors are coverage (Section 4.2) and damage to legitimate text (Section 4.5).

5 Discussion & Evaluation

5.1 Answering the research questions

RQ1. Five of six attacks work well: TokenBreak and AdvTok on 96.2% of eligible inputs, the other Unicode variants between 85.9% and 91.0%. It is not a borderline effect, since the strongest two still win 96.1% of the time on the 128 messages the classifier was most sure about. The sixth fails because its perturbation never changes the token identifiers, so it never reaches the model at all.

RQ2. No defence provides broad coverage across the tested attack families, and one interaction makes matters worse. Canonical enforcement removes AdvTok, NFKC repair removes the tested compatibility and homoglyph variants, and sanitisation removes the reorder variant. TokenBreak remains the principal residual gap: tokenizer translation produces the largest reduction but still leaves 46.5% defended ASR, while global CPT provides a smaller partial reduction. Tokenizer translation turns a 0% attack into a 13.4% one. The paired protocol is what makes these readings trustworthy, and it changes one outright:

translation keeps only 81.4% of the clean toxic messages, so almost a fifth of the positive class is lost before the attacker acts. A check on benign text alone would never show it, because translation preserves 98.8% of benign decisions.

RQ3. The cost is real and unevenly distributed. Global fragmentation filtering blocks every tested web address and non-English input; tokenizer translation rewrites almost every structured or multilingual input. Against AdvTok alone, global fragmentation filtering appears highly effective; the stress corpus shows why that security result is insufficient for deployment (Appendix C, Figure 7). It supports Broken-Token's idea that heavy fragmentation is cheaply detectable [8], and shows the same statistic treating non-English and structured text as anomalous once its threshold is fitted to English prose, which echoes Axelsson's warning that false alarms can make an otherwise useful detector operationally unsuitable [13]. The attacks were never re-optimised against the filter, so its strength is evidence against these frozen attacks, not a guarantee.

5.2 Achievement of objectives

O1 to O5 in Table 1 were met in full, evidenced by Sections 3.5.1, 3.5.2, 4.2, 4.5 and 4.6. O6 is met with one qualification: the environment snapshot fixes the final evidence state rather than every historical run, and the BPE/Unigram groundwork stops at clean baselines. No objective was rewritten after the results were seen.

5.3 Threats to validity

Several limitations change the bound on the claim rather than just lowering confidence in it, so Appendix A, Table 7 pairs each with the bound it imposes.

Two qualifications matter most. A zero-success cell holds only for the tested variant under the frozen threat model, and where the generator and the repair share a character family it says more about implementation correctness than robustness. Separately, a defender-aware attacker could optimise against the fragmentation filter or the preprocessing stage. That experiment was not run, and no claim here survives it untested.

5.4 Reflection

This study followed a large preliminary programme on cross-tokenizer disagreement, which froze 134,217 LMSYS and 75,839 WildChat user turns and produced a 293,034-row paired corpus from 48,839 deduplicated prompts. None is pooled into the present sample, because the models, attacks and question all differ.

That detector hypothesis was tested under a pre-registered comparison and came back "NOT SUPERIOR" to the fragmentation baseline, because the paired interval for the difference included zero. I changed the research question rather than the success criterion after seeing the data, which cost me most of a term and is the decision I would defend first. The corrections in Section 3.5.4 follow the same pattern: each began with me checking my own implementation against the source and finding it wanting.

Those corrections mark the limits of what I can defend. I can reproduce the tokenizer mechanics, the paired design and every frozen analysis. I cannot claim adaptive robustness, having run no adaptive attacks, nor cross-tokenizer generalisation, the BPE and Unigram attack stage never having been executed. Writing those limits into the result is more useful than dressing unfinished work as a stronger conclusion.

6 Conclusion

This dissertation put six tokenizer-level attacks against seven defences on one WordPiece toxicity classifier, using 936 attack instances, 6,552 paired evaluations, 5,250 ordinary clean controls and a 600-input legitimate stress set. Five attacks succeeded on most inputs; the sixth was neutralised by the classifier's own preprocessing, providing a concrete explanation for its failure on this victim and distinguishing between explanations previously proposed by Boucher et al. [1].

Defence performance was attack-specific throughout, and utility reordered the ranking. Tokenizer translation gave the largest TokenBreak reduction and still left 46.5% of attacks working, while destroying 18.6% of the classifier's correct toxic decisions. Global fragmentation filtering was the best detector of AdvTok in the study and preserved only 44.8% of complex legitimate input. One defence raised an inert attack to 13.4%. Several controls reached zero, but the attack-aware ones among them measure implementation correctness rather than coverage.

The conclusion I can defend is narrow. Tokenizer defences should be judged against the attacks and the legitimate input distribution of the deployment they are meant for, with clean and attacked messages processed under the same control; a low success rate against one attack is not a reason to adopt a preprocessing policy. Future work should add defence-aware adaptive attacks, repeat the matrix on controlled BPE and Unigram victims, extend it to the guard-model baseline excluded in Section 1.4, and apply these defences to the frozen LMSYS and WildChat traffic.

References

[1] Nicholas Boucher, Ilia Shumailov, Ross Anderson, and Nicolas Papernot. 2022. Bad Characters: Imperceptible NLP Attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. DOI: https://doi.org/10.1109/SP46214.2022.9833641. Accessed 8 August 2026.

[2] Renato Lui Geh, Zilei Shao, and Guy Van den Broeck. 2025. Adversarial Tokenization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 20738–20765. DOI: https://doi.org/10.18653/v1/2025.acl-long.1012. Accessed 8 August 2026.

[3] Kasimir Schulz, Kenneth Yeung, and Kieran Evans. 2025. TokenBreak: Bypassing Text Classification Models Through Token Manipulation. arXiv:2506.07948 [cs.LG]. Preprint. https://arxiv.org/abs/2506.07948. Accessed 8 August 2026.

[4] Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. 2018. HotFlip: White-Box Adversarial Examples for Text Classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 31–36. DOI: https://doi.org/10.18653/v1/P18-2006. Accessed 8 August 2026.

[5] Jinfeng Li, Shouling Ji, Tianyu Du, Bo Li, and Ting Wang. 2019. TextBugger: Generating Adversarial Text Against Real-world Applications. In *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS 2019)*. DOI: https://doi.org/10.14722/ndss.2019.23138. Accessed 8 August 2026.

[6] Hossein Hosseini, Sreeram Kannan, Baosen Zhang, and Radha Poovendran. 2017. Deceiving Google's Perspective API Built for Detecting Toxic Comments. arXiv:1702.08138 [cs.LG]. Preprint. https://arxiv.org/abs/1702.08138. Accessed 8 August 2026.

[7] Danish Pruthi, Bhuwan Dhingra, and Zachary C. Lipton. 2019. Combating Adversarial Misspellings with Robust Word Recognition. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 5582–5591. DOI: https://doi.org/10.18653/v1/P19-1561. Accessed 8 August 2026.

[8] Shaked Zychlinski and Yuval Kainan. 2025. Broken-Token: Filtering Obfuscated Prompts by Counting Characters-Per-Token. arXiv:2510.26847 [cs.CR]. Preprint. https://arxiv.org/abs/2510.26847. Accessed 8 August 2026.

[9] Mark Davis and Michel Suignard (eds.). 2025. Unicode Technical Standard #39: Unicode Security Mechanisms, Version 17.0.0. Unicode Consortium. https://www.unicode.org/reports/tr39/. Accessed 8 August 2026.

[10] Nicholas Carlini, Anish Athalye, Nicolas Papernot, Wieland Brendel, Jonas Rauber, Dimitris Tsipras, Ian Goodfellow, Aleksander Madry, and Alexey Kurakin. 2019. On Evaluating Adversarial Robustness. arXiv:1902.06705 [cs.LG]. DOI: https://doi.org/10.48550/arXiv.1902.06705. Accessed 8 August 2026.

[11] Florian Tramèr, Nicholas Carlini, Wieland Brendel, and Aleksander Madry. 2020. On Adaptive Attacks to Adversarial Example Defenses. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. https://proceedings.neurips.cc/paper/2020/hash/11f38f8ecd71867b42433548d1078e38-Abstract.html. Accessed 8 August 2026.

[12] Nicolas Papernot, Patrick McDaniel, Arunesh Sinha, and Michael P. Wellman. 2018. SoK: Security and Privacy in Machine Learning. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, 399–414. DOI: https://doi.org/10.1109/EuroSP.2018.00035. Accessed 8 August 2026.

[13] Stefan Axelsson. 2000. The base-rate fallacy and the difficulty of intrusion detection. *ACM Transactions on Information and System Security* 3, 3, 186–205. DOI: https://doi.org/10.1145/357830.357849. Accessed 8 August 2026.

[14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT 2019 (Volume 1: Long and Short Papers)*, 4171–4186. DOI: https://doi.org/10.18653/v1/N19-1423. Accessed 8 August 2026.

[15] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1715–1725. DOI: https://doi.org/10.18653/v1/P16-1162. Accessed 8 August 2026.

[16] Taku Kudo and John Richardson. 2018. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. In *Proceedings of EMNLP 2018: System Demonstrations*, 66–71. DOI: https://doi.org/10.18653/v1/D18-2012. Accessed 8 August 2026.

[17] Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. 2021. How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models. In *Proceedings of ACL-IJCNLP 2021 (Volume 1: Long Papers)*, 3118–3135. DOI: https://doi.org/10.18653/v1/2021.acl-long.243. Accessed 8 August 2026.

[18] Craig W. Schmidt, Varshini Reddy, Haoran Zhang, Alec Alameddine, Omri Uzan, Yuval Pinter, and Chris Tanner. 2024. Tokenization Is More Than Compression. In *Proceedings of EMNLP 2024*, 678–702. DOI: https://doi.org/10.18653/v1/2024.emnlp-main.40. Accessed 8 August 2026.

[19] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. 2020. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. In *Proceedings of ACL 2020*, 4902–4912. DOI: https://doi.org/10.18653/v1/2020.acl-main.442. Accessed 8 August 2026.

[20] Lucas Dixon, John Li, Jeffrey Sorensen, Nithum Thain, and Lucy Vasserman. 2018. Measuring and Mitigating Unintended Bias in Text Classification. In *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society (AIES '18)*, 67–73. DOI: https://doi.org/10.1145/3278721.3278729. Accessed 8 August 2026.

[21] Daniel Borkan, Lucas Dixon, Jeffrey Sorensen, Nithum Thain, and Lucy Vasserman. 2019. Nuanced Metrics for Measuring Unintended Bias with Real Data for Text Classification. In *Companion Proceedings of the 2019 World Wide Web Conference (WWW '19 Companion)*, 491–500. DOI: https://doi.org/10.1145/3308560.3317593. Accessed 8 August 2026.

[22] Bradley Efron. 1979. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics* 7, 1, 1–26. DOI: https://doi.org/10.1214/aos/1176344552. Accessed 8 August 2026.

[23] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association* 22, 158, 209–212. DOI: https://doi.org/10.1080/01621459.1927.10502953. Accessed 8 August 2026.
[24] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108 [cs.CL]. DOI: https://doi.org/10.48550/arXiv.1910.01108. Accessed 8 August 2026.
[25] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer Sentinel Mixture Models. In *ICLR 2017*. Introduces the WikiText datasets. arXiv:1609.07843. DOI: https://doi.org/10.48550/arXiv.1609.07843. Accessed 8 August 2026.
[26] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. arXiv:1909.09436 [cs.LG]. https://arxiv.org/abs/1909.09436. Accessed 8 August 2026.
[27] Dorottya Demszky, Dana Movshovitz-Attias, Jeongwoo Ko, Alan Cowen, Gaurav Nemade, and Sujith Ravi. 2020. GoEmotions: A Dataset of Fine-Grained Emotions. In *Proceedings of ACL 2020*, 4040–4054. DOI: https://doi.org/10.18653/v1/2020.acl-main.372. Accessed 8 August 2026.
[28] Naman Goyal, Cynthia Gao, Vishrav Chaudhary, Peng-Jen Chen, Guillaume Wenzek, Da Ju, Sanjana Krishnan, Marc'Aurelio Ranzato, Francisco Guzmán, and Angela Fan. 2022. The Flores-101 Evaluation Benchmark for Low-Resource and Multilingual Machine Translation. *Transactions of the Association for Computational Linguistics* 10, 522–538. DOI: https://doi.org/10.1162/tacl_a_00474. Accessed 8 August 2026.
[29] NLLB Team. 2024. Scaling neural machine translation to 200 languages. *Nature* 630, 8018, 841–846. DOI: https://doi.org/10.1038/s41586-024-07335-x. Accessed 8 August 2026.
[30] Martin Pan (martin-ha). 2021. `martin-ha/toxic-comment-model`. Hugging Face model repository and model card. https://huggingface.co/martin-ha/toxic-comment-model. Accessed 8 August 2026.
[31] Conversation AI (Jigsaw and Google). 2018. Toxic Comment Classification Challenge. Kaggle competition and dataset. https://www.kaggle.com/competitions/jigsaw-toxic-comment-classification-challenge. Accessed 8 August 2026.
[32] Lorenzo Cavallaro and Emiliano De Cristofaro. 2023. Security and Privacy of AI Knowledge Guide, Issue 1.0.0. CyBOK supplementary guide, University of Bristol. https://www.cybok.org/media/downloads/Security_Privacy_AI_KG_v1.0.0.pdf. Accessed 8 August 2026.
[33] Gianluca Stringhini. 2021. Adversarial Behaviours Knowledge Area, Version 1.0.1. CyBOK. https://www.cybok.org/media/downloads/Adversarial_Behaviours_v1.0.1.pdf. Accessed 8 August 2026.
[34] Frank Piessens. 2021. Software Security Knowledge Area, Version 1.0.1. CyBOK. https://www.cybok.org/media/downloads/Software_Security_v1.0.1.pdf. Accessed 8 August 2026.

A Supporting tables

Table 7. Threats to validity and the bound each places on interpretation.

Threat	Bound on the claim
Single primary victim	Conclusions are within-victim. The clean BPE/Unigram arm is supporting evidence; no cross-family generalisation is claimed.
Possible train-evaluation overlap	The model card reports Jigsaw-derived fine-tuning and the evaluation samples Jigsaw. Exact overlap is unknown, so no out-of-distribution generalisation is claimed and the direction of any bias is unknown.
Adapted attacks and defences	AdvTok, the Unicode objective, CPT thresholding and the TokenBreak deep check all contain project changes. Results describe these implemented variants, not the source methods everywhere.
No adaptive attacks	Attacks were generated against the undefended victim and replayed. Defence results, CPT above all, may overestimate performance against a defence-aware attacker [10, 11].
Attack-aware Unicode controls	Homoglyph generation and confusable repair share a mapping family, and compatibility characters were chosen for NFKC behaviour, so those zero-ASR cells are implementation checks.

B Disclosed repair to the TokenBreak procedure

The published TokenBreak deep check raises its confidence threshold and then re-enters *BreakPrompt*, whose printed procedure initialises that threshold again. The implementation therefore preserves the raised threshold explicitly and repeats the attack iteratively, with an upper bound of 0.9999. Each retry starts from the original prompt, matching the implementation used to generate the frozen results.

Algorithm 2: Bounded TokenBreak deep-check repair. Each retry runs *BreakPrompt* on the original prompt using the increased confidence threshold. The repair preserves the intended threshold escalation that is lost when the published procedure reinitialises the threshold.

Input: prompt x, classifier f
Output: attacked prompt x', final threshold τ

if $f(x) \neq$ toxic **then**
  return $(x, 0.995)$
end
$\tau \leftarrow 0.995$
$x' \leftarrow \text{BreakPrompt}(x, \tau)$
while $f(x') =$ toxic ***and*** $\tau < 0.9999$ **do**
  $\tau \leftarrow \tau + 0.0001$
  $x' \leftarrow \text{BreakPrompt}(x, \tau)$
end
return (x', τ)

C Supplementary visualisations

Every figure is placed here so that it can be printed large enough to read. Each is referenced from the section of the main text that discusses it, and every value shown is also stated numerically in Sections 4 and 5.

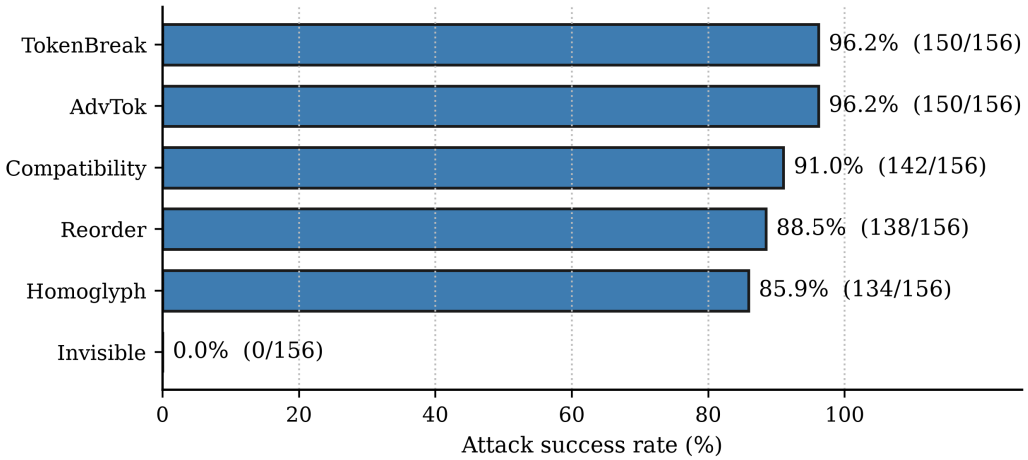

Fig. 3. Attack success against the undefended classifier, as a percentage of the 156 eligible toxic inputs with counts alongside. TokenBreak and AdvTok tie at the top; the invisible-character variant fails completely because its token identifiers are identical to the clean ones.

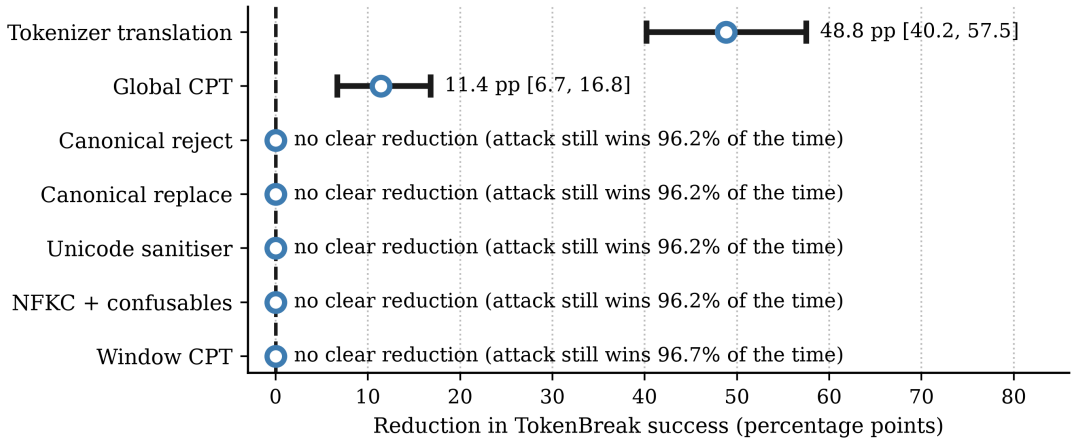

Fig. 4. How much each defence reduces TokenBreak, in percentage points, with 95% bootstrap intervals. Only two defences move the number at all, and the better of the two still leaves the attacker succeeding roughly half the time.

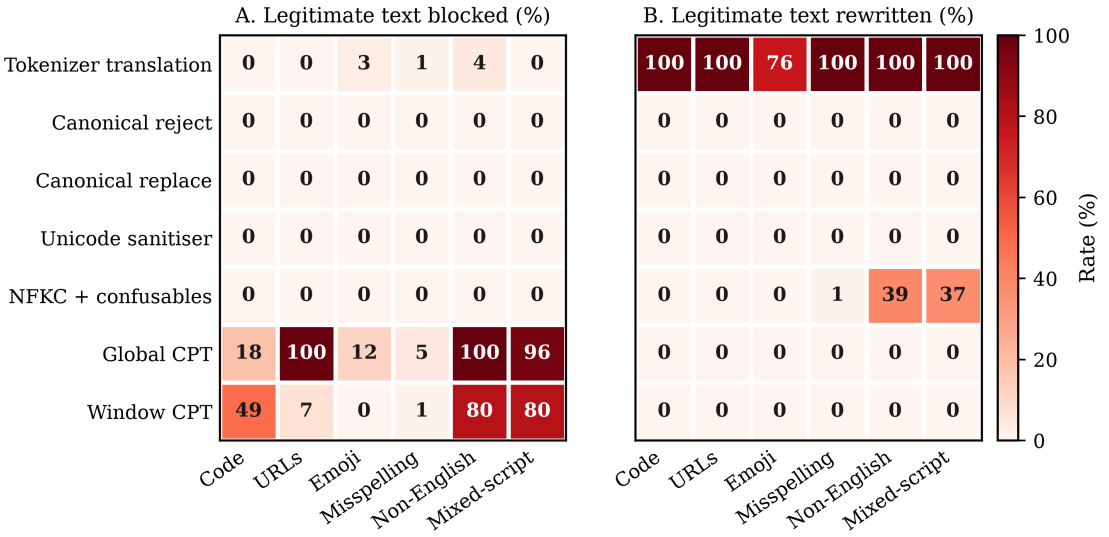

Fig. 5. What each defence does to 600 legitimate inputs. Panel A shows how much of each category the defence blocks outright; Panel B shows how often it leaves the decision alone but rewrites the text. Darker red always means the defence interfered more, so the two fragmentation filters and tokenizer translation stand out for opposite reasons.

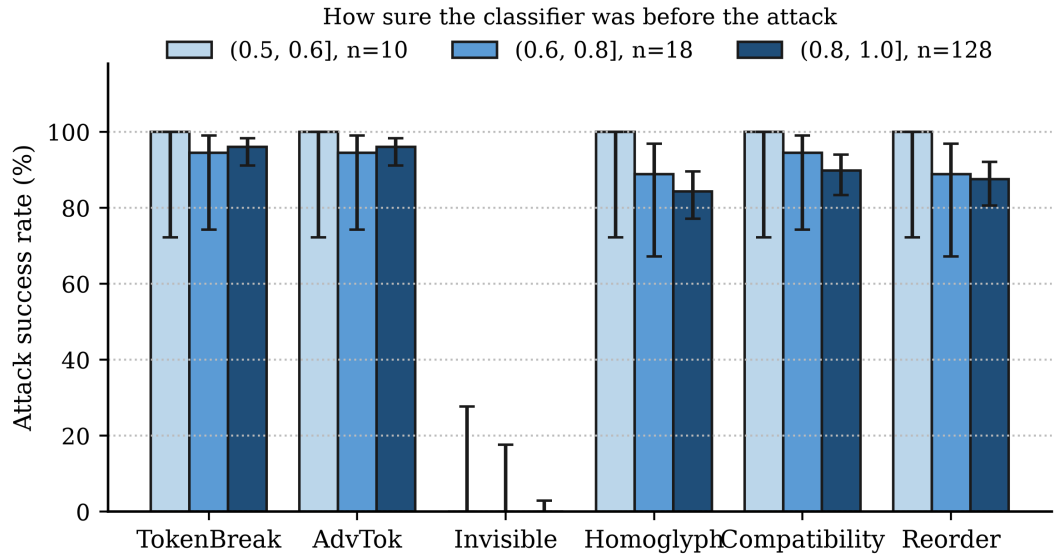

Fig. 6. Attack success split by how confident the clean classifier was, with 95% Wilson intervals. The rightmost bar of each group holds 128 of the 156 inputs, so the high success rates are not coming from borderline cases.

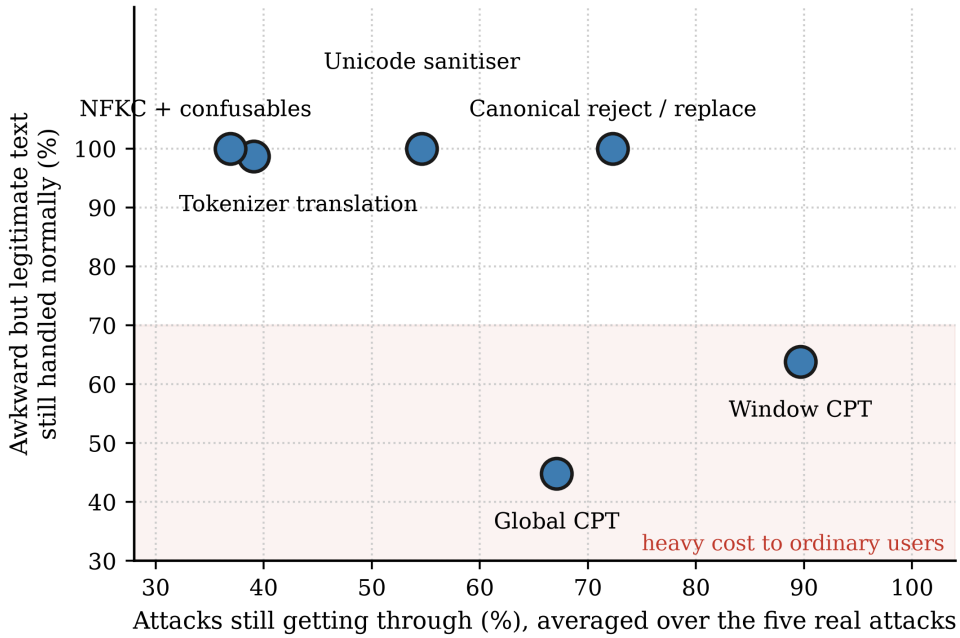

Fig. 7. The trade-off in one picture. Left is better on the horizontal axis (fewer attacks get through) and higher is better on the vertical one (less damage to ordinary text). The two fragmentation filters sit in the shaded region: they are the most disruptive controls in the study and are not even the best at stopping attacks. Canonical reject and canonical replace land on exactly the same point.

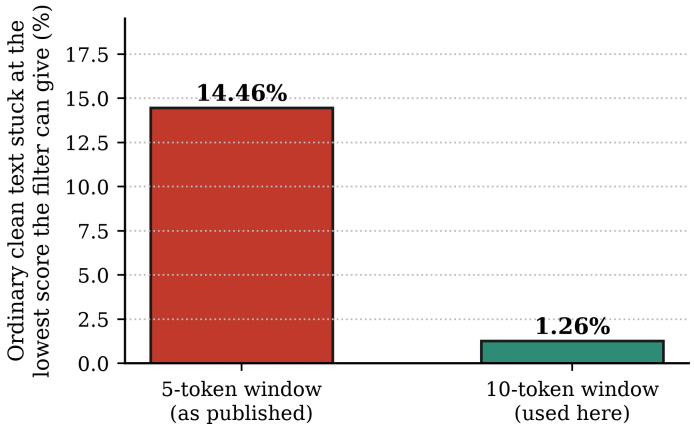

Fig. 8. Why the published window size had to change. With the five-token window, 14.46% of ordinary clean text scores exactly the lowest value the filter can produce, so a threshold cannot separate anything below it. Widening the window to ten tokens drops that to 1.26% and makes the threshold meaningful.

D Artefact and Source Code

The complete project artefact and source code are available at:

https://github.com/ApoorvMani/Dissertation_and_Artefact

The repository contains the implementation, supporting scripts, frozen result artefacts, and materials required to inspect and reproduce the evaluation described in this dissertation.